TASK 3 – Define the Constants

Centralised tariff definitions:

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS) {
        paise += (SLAB2_UNITS - SLAB1_UNITS) *
RATE2_PAISE;
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
    } else if (units > SLAB1_UNITS) {
        paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
    }
```

```
    /* The first 100 units are free, so no energy charge is added for
them. */
    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = (int)(sizeof tests / sizeof tests[0]);
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Expected output:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

#define, const and enum:

Feature	Meaning	Best use here
#define	Preprocessor replaces the token before compilation; it has no C type and no storage of its own.	Simple tariff numbers and compile-time configuration.
const variable	A typed object that code is not allowed to modify through that identifier; scope and debugger visibility are clearer.	Typed values, especially local or grouped configuration.
enum	Defines named integer constants for a finite set of alternatives.	Customer categories or named states, not fractional tariff rates.

A macro is easy to update in one place but does not provide normal type checking and may be less convenient to inspect in a debugger. A const object is type-checked and has scope; it may or may not require storage after optimisation. An enum is best for discrete integer choices. Runtime and memory effects of these choices are usually negligible for simple tariff constants; correctness and maintainability are more important.

Changing RATE2_PAISE from 225 to 230 changes every applicable bill consistently because the rate is defined once. After changing it, recompile and rerun all boundary tests.